# Session 07 — Comprehension / Theory

## Q15 — Comparison Table

| Feature | Static Binding (new) | Dynamic Binding (override) |
|---|---|---|
| Keyword in base | No special keyword needed — any regular method can be hidden | virtual (or abstract) |
| Keyword in derived | new | override |
| Resolved at | Compile time — based on the declared (reference) type of the variable | Runtime — based on the actual (real) type of the object |
| Behavior via base reference | Calls the base class's version, even if the object is really a derived type | Calls the derived class's overridden version, even though the reference is typed as the base class |

## Q16 — Why does override require virtual, but new works on any method?

override is a contract about runtime polymorphism: it tells the compiler "at runtime, when this method is called through any reference to this object, dynamically dispatch to this implementation, no matter what the reference's declared type is." Making this behavior work requires the base method to have already opted into the virtual dispatch mechanism (a hidden "vtable"-style lookup) by being marked virtual (or abstract) in the first place.

If C# allowed override on a plain, non-virtual method, there would be no dispatch table to hook into, and the promise of "polymorphic behavior at runtime" could not actually be honored — so the compiler enforces this requirement to prevent writing code that looks polymorphic but silently isn't.

new, on the other hand, doesn't promise any runtime polymorphism at all — it simply says "in this class, define a brand-new method with the same name/signature, completely unrelated to the base class's method of the same name." Since it doesn't rely on any dynamic dispatch mechanism (it's resolved purely by the compile-time declared type of the reference), it doesn't need the base method to be virtual — it works identically whether the base method is virtual, non-virtual, or even something totally unrelated.

That's exactly why new can be applied to hide any method, while override can only be applied to methods the base class has explicitly made overridable.